

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Đồ Án Cuối Kỳ

Object Detection Application

23120245 - NGUYỄN QUANG DUY
23120258 - LƯU TRỌNG HIẾU
23120260 - VĂN ĐÌNH HIẾU

Môn: Học thống kê

Thành phố Hồ Chí Minh – 2026

Mục lục

1	Giới thiệu	2
2	Chuẩn bị dữ liệu	2
2.1	Nguồn dữ liệu	2
2.2	Định dạng dữ liệu	3
2.3	Phân chia dữ liệu train/validation/test	3
2.4	Cấu trúc thư mục dữ liệu	3
3	Các mô hình thử nghiệm	4
3.1	Faster R-CNN	4
3.2	YOLOv8	4
3.3	Deformable DETR	4
4	Phương pháp đánh giá	5
5	Kết quả thực nghiệm	5
5.1	Bảng kết quả tổng quan	5
5.2	So sánh về độ chính xác	5
5.3	So sánh về tốc độ xử lý	6
5.4	So sánh kích thước mô hình	6
6	Nhận xét ưu điểm và nhược điểm của từng mô hình	6
6.1	Faster R-CNN	6
6.1.1	Ưu điểm	6
6.1.2	Nhược điểm	7
6.1.3	Khả năng áp dụng thực tế	7
6.2	YOLOv8	7
6.2.1	Ưu điểm	7
6.2.2	Nhược điểm	7
6.2.3	Khả năng áp dụng thực tế	7
6.3	Deformable DETR	8
6.3.1	Ưu điểm	8
6.3.2	Nhược điểm	8
6.3.3	Khả năng áp dụng thực tế	8
7	Lựa chọn mô hình cuối cùng	8
8	Xây dựng ứng dụng web	9
9	Kết luận	9

1 Giới thiệu

Object Detection là một trong những bài toán quan trọng trong lĩnh vực thị giác máy tính, với mục tiêu không chỉ xác định sự xuất hiện của đối tượng trong ảnh mà còn định vị vị trí của đối tượng đó thông qua bounding box. Trong đồ án này, nhóm thực hiện xây dựng một hệ thống phát hiện đối tượng trong bối cảnh giao thông, bao gồm các lớp như người, xe đạp, xe hơi, xe máy, xe buýt, xe tải, đèn giao thông và biển báo dừng.

Mục tiêu chính của project gồm hai phần. Thứ nhất là phần thực nghiệm, trong đó nhóm tiến hành chuẩn bị dữ liệu, huấn luyện và đánh giá nhiều kiến trúc object detection khác nhau. Thứ hai là phần ứng dụng, trong đó nhóm lựa chọn mô hình phù hợp nhất để tích hợp vào một web application cho phép người dùng upload ảnh hoặc sử dụng webcam để phát hiện đối tượng.

Trong project này, ba mô hình được thử nghiệm gồm Faster R-CNN, YOLOv8 và Deformable DETR. Đây là ba đại diện cho ba hướng tiếp cận khác nhau trong object detection: mô hình two-stage detector, mô hình one-stage detector và mô hình dựa trên Transformer.

2 Chuẩn bị dữ liệu

2.1 Nguồn dữ liệu

Dữ liệu sử dụng trong project được xây dựng từ tập COCO, trong đó nhóm chọn ra các ảnh liên quan đến bối cảnh giao thông. Các đối tượng được lựa chọn là những lớp thường xuất hiện trong môi trường đường phố và có ý nghĩa thực tế đối với bài toán nhận diện giao thông.

Tập dữ liệu gồm 8 lớp đối tượng:

- person
- bicycle
- car
- motorcycle
- bus
- truck
- traffic light
- stop sign

Các lớp này được lựa chọn vì chúng đại diện cho những thành phần phổ biến trong giao thông đô thị. Việc giới hạn số lớp giúp bài toán tập trung hơn, giảm độ phức tạp so với việc huấn luyện trên toàn bộ tập COCO, đồng thời vẫn đảm bảo tính ứng dụng thực tế.

2.2 Định dạng dữ liệu

Do các mô hình khác nhau yêu cầu định dạng annotation khác nhau, dữ liệu được chuẩn bị theo hai định dạng chính:

- YOLO format: dùng cho mô hình YOLOv8.
- COCO format: dùng cho Faster R-CNN và Deformable DETR.

Với YOLO format, mỗi ảnh có một file nhãn tương ứng, trong đó mỗi dòng biểu diễn một bounding box theo dạng:

```
class_id x_center y_center width height
```

Các giá trị tọa độ được chuẩn hóa theo kích thước ảnh.

Với COCO format, annotation được lưu trong các file JSON, bao gồm thông tin ảnh, danh sách categories và danh sách annotations. Định dạng này phù hợp với các mô hình sử dụng thư viện PyTorch/TorchVision hoặc Hugging Face Transformers.

2.3 Phân chia dữ liệu train/validation/test

Sau khi xử lý và chuẩn hóa annotation, dữ liệu được chia thành ba tập: train, validation và test theo tỉ lệ 70/15/15.

Cụ thể:

- Tập train: 3500 ảnh, chiếm khoảng 70% dữ liệu.
- Tập validation: 749 ảnh, chiếm khoảng 15% dữ liệu (1 ảnh bị lỗi đến từ dataset trong quá trình tải).
- Tập test: 750 ảnh, chiếm khoảng 15% dữ liệu.

Tập train được sử dụng để huấn luyện mô hình. Trong quá trình này, các tham số của mô hình được cập nhật dựa trên dữ liệu huấn luyện.

Tập validation được sử dụng để theo dõi khả năng tổng quát hóa của mô hình trong quá trình huấn luyện. Kết quả trên tập validation giúp lựa chọn checkpoint tốt nhất, đồng thời hạn chế tình trạng overfitting.

Tập test được sử dụng để đánh giá cuối cùng sau khi quá trình huấn luyện hoàn tất. Các kết quả so sánh giữa các mô hình trong báo cáo được lấy trên tập test nhằm đảm bảo tính công bằng.

2.4 Cấu trúc thư mục dữ liệu

Cấu trúc dữ liệu sau khi xử lý như sau:

```
data/processed/  
├── train/  
│   ├── images/  
│   └── labels/  
└── val/  
    └── images/
```

```
├── labels/
├── test/
│   ├── images/
│   ├── labels/
│   └── annotations/
├── train.json
├── val.json
└── test.json
```

Trong đó, thư mục `images` chứa ảnh đầu vào, thư mục `labels` chứa nhãn theo định dạng YOLO, còn thư mục `annotations` chứa các file JSON theo định dạng COCO.

3 Các mô hình thử nghiệm

3.1 Faster R-CNN

Faster R-CNN là mô hình object detection thuộc nhóm two-stage detector. Mô hình này gồm hai giai đoạn chính.

Ở giai đoạn đầu, Region Proposal Network được sử dụng để đề xuất các vùng có khả năng chứa đối tượng. Ở giai đoạn thứ hai, các vùng đề xuất được đưa qua head phân loại để xác định lớp đối tượng và tính chỉnh bounding box.

Ưu điểm lớn của Faster R-CNN là độ chính xác cao và khả năng phát hiện đối tượng ổn định. Tuy nhiên, do phải thực hiện hai giai đoạn xử lý, tốc độ suy luận của mô hình thường chậm hơn so với các mô hình one-stage như YOLO.

Trong project này, Faster R-CNN được sử dụng như một mô hình mạnh về độ chính xác và là baseline quan trọng để so sánh với các kiến trúc khác.

3.2 YOLOv8

YOLOv8 là mô hình thuộc nhóm one-stage detector. Khác với Faster R-CNN, YOLOv8 dự đoán trực tiếp bounding box, class và confidence score trong một lần forward qua mạng.

Ưu điểm nổi bật của YOLOv8 là tốc độ xử lý nhanh, kích thước mô hình nhỏ và dễ triển khai trong các ứng dụng thực tế. Đây là mô hình phù hợp với các bài toán yêu cầu phản hồi nhanh hoặc gần realtime.

Tuy nhiên, trong một số trường hợp, đặc biệt với các đối tượng nhỏ, bị che khuất hoặc xuất hiện trong bối cảnh phức tạp, YOLOv8 có thể cho độ chính xác thấp hơn so với Faster R-CNN.

3.3 Deformable DETR

Deformable DETR là mô hình object detection dựa trên Transformer, được cải tiến từ DETR. Mô hình sử dụng cơ chế attention để học quan hệ toàn cục trong ảnh, đồng thời sử dụng deformable attention nhằm cải thiện hiệu quả tính toán và khả năng hội tụ.

Đây là kiến trúc hiện đại, có tiềm năng tốt khi được huấn luyện trên dữ liệu lớn và trong điều kiện tài nguyên tính toán mạnh. Tuy nhiên, so với Faster R-CNN và YOLOv8, Deformable DETR có độ phức tạp cao hơn, quá trình huấn luyện khó hơn và cần nhiều tài nguyên hơn.

Trong project này, Deformable DETR được thử nghiệm nhằm đại diện cho hướng tiếp cận Transformer-based trong object detection.

4 Phương pháp đánh giá

Các mô hình được đánh giá trên tập test gồm 750 ảnh. Việc đánh giá được thực hiện trên CPU để đảm bảo cùng điều kiện phần cứng khi so sánh tốc độ xử lý.

Các chỉ số chính được sử dụng gồm:

- Precision: tỉ lệ dự đoán đúng trong tổng số dự đoán dương tính.
- Recall: tỉ lệ đối tượng thật được mô hình phát hiện đúng.
- F1-score: trung bình điều hòa giữa precision và recall.
- mAP@0.5: mean Average Precision tại ngưỡng IoU bằng 0.5.
- mAP@0.5:0.95: mean Average Precision trung bình trên nhiều ngưỡng IoU từ 0.5 đến 0.95.
- FPS: số frame xử lý được trong một giây.
- Model size: kích thước trọng số mô hình.

Trong đó, mAP@0.5 và mAP@0.5:0.95 là hai chỉ số quan trọng để đánh giá độ chính xác tổng thể của mô hình object detection. FPS được sử dụng để đánh giá khả năng ứng dụng thực tế, đặc biệt trong các hệ thống cần xử lý nhanh.

5 Kết quả thực nghiệm

5.1 Bảng kết quả tổng quan

Mô hình	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	FPS
Faster R-CNN	0.3727	0.4718	0.4164	0.6062	0.3727	0.37
YOLOv8	0.4492	0.3828	0.4133	0.3764	0.2406	23.81
Deformable DETR	0.0744	0.1823	0.1057	0.1604	0.0744	0.62

Bảng 1: So sánh kết quả đánh giá giữa các mô hình trên tập test

5.2 So sánh về độ chính xác

Dựa trên bảng kết quả, Faster R-CNN đạt kết quả tốt nhất về độ chính xác. Mô hình đạt mAP@0.5 là 0.6062 và mAP@0.5:0.95 là 0.3727, cao nhất trong ba mô hình được thử nghiệm. Điều này cho thấy Faster R-CNN có khả năng phát hiện và định vị đối tượng tốt hơn trên tập dữ liệu hiện tại.

YOLOv8 đạt precision trung bình cao nhất với giá trị 0.4492, cho thấy khi mô hình đưa ra dự đoán, tỉ lệ dự đoán đúng là tương đối tốt. Tuy nhiên, recall và mAP của

YOLOv8 thấp hơn Faster R-CNN. Điều này cho thấy YOLOv8 có thể bỏ sót một số đối tượng hoặc định vị bounding box chưa chính xác bằng Faster R-CNN.

Deformable DETR có kết quả thấp nhất trong ba mô hình. Precision, recall, F1-score và mAP đều thấp hơn đáng kể so với Faster R-CNN và YOLOv8. Điều này có thể xuất phát từ việc mô hình Transformer-based thường cần nhiều dữ liệu hơn, thời gian huấn luyện lâu hơn và quá trình tinh chỉnh phức tạp hơn để đạt hiệu quả tốt.

5.3 So sánh về tốc độ xử lý

Về tốc độ xử lý, YOLOv8 vượt trội so với hai mô hình còn lại. Mô hình đạt khoảng 23.81 FPS trên CPU, cho thấy khả năng xử lý nhanh và phù hợp với các ứng dụng yêu cầu phản hồi gần realtime.

Faster R-CNN đạt khoảng 0.37 FPS trên CPU, thấp nhất trong ba mô hình. Điều này phản ánh đặc điểm của mô hình two-stage detector: mặc dù có độ chính xác cao, quá trình suy luận phức tạp hơn và tốn nhiều thời gian hơn.

Deformable DETR đạt khoảng 0.62 FPS trên CPU, nhanh hơn Faster R-CNN nhưng vẫn rất thấp so với YOLOv8. Mặc dù kiến trúc Transformer có tiềm năng mạnh, việc suy luận trên CPU không phải là điều kiện lý tưởng cho mô hình này.

5.4 So sánh kích thước mô hình

Mô hình	Kích thước model (MB)	Nhận xét
Faster R-CNN	158.17	Lớn, tốn tài nguyên khi triển khai
YOLOv8	5.94	Nhỏ, dễ triển khai
Deformable DETR	155.81	Lớn, yêu cầu tài nguyên cao

Bảng 2: So sánh kích thước mô hình

YOLOv8 có kích thước nhỏ nhất, chỉ khoảng 5.94 MB. Đây là lợi thế lớn khi triển khai mô hình vào web application hoặc các môi trường có tài nguyên hạn chế.

Faster R-CNN và Deformable DETR có kích thước lớn hơn nhiều, lần lượt khoảng 158.17 MB và 155.81 MB. Điều này khiến việc triển khai hai mô hình này cần nhiều bộ nhớ hơn và thời gian load model lâu hơn.

6 Nhận xét ưu điểm và nhược điểm của từng mô hình

6.1 Faster R-CNN

6.1.1 Ưu điểm

Faster R-CNN đạt độ chính xác cao nhất trong các mô hình thử nghiệm. Với mAP@0.5 đạt 0.6062 và mAP@0.5:0.95 đạt 0.3727, mô hình cho thấy khả năng phát hiện đối tượng tốt trên tập test.

Do sử dụng cơ chế two-stage, Faster R-CNN có khả năng đề xuất vùng chứa đối tượng trước khi phân loại, giúp mô hình định vị bounding box chính xác hơn. Điều này đặc biệt hữu ích trong các bài toán cần độ chính xác cao.

Ngoài ra, Faster R-CNN là một kiến trúc phổ biến, có tính ổn định cao và dễ giải thích trong báo cáo thực nghiệm. Mô hình phù hợp để làm baseline mạnh khi so sánh với các kiến trúc khác.

6.1.2 Nhược điểm

Nhược điểm lớn nhất của Faster R-CNN là tốc độ xử lý chậm. Trong kết quả thực nghiệm, mô hình chỉ đạt khoảng 0.37 FPS trên CPU. Tốc độ này không phù hợp với các ứng dụng realtime nếu không có tối ưu hoặc không sử dụng GPU.

Bên cạnh đó, mô hình có kích thước lớn, khoảng 158.17 MB, khiến việc load model và triển khai ứng dụng tiêu tốn nhiều tài nguyên hơn YOLOv8.

Quá trình huấn luyện Faster R-CNN cũng phức tạp hơn so với YOLOv8. Người triển khai cần xử lý đúng định dạng COCO, mapping class và cấu trúc target phù hợp với TorchVision.

6.1.3 Khả năng áp dụng thực tế

Faster R-CNN phù hợp với các ứng dụng ưu tiên độ chính xác hơn tốc độ, ví dụ như phân tích ảnh offline, hệ thống kiểm tra ảnh sau khi chụp hoặc các ứng dụng không yêu cầu realtime.

Trong project này, nhóm lựa chọn Faster R-CNN làm mô hình chính vì mô hình đạt kết quả chính xác cao nhất trên tập test. Tuy nhiên, khi triển khai thực tế, cần lưu ý rằng mô hình nên được chạy trên GPU hoặc cần có cơ chế tối ưu để cải thiện tốc độ xử lý.

6.2 YOLOv8

6.2.1 Ưu điểm

YOLOv8 có tốc độ xử lý nhanh nhất trong ba mô hình. Với khoảng 23.81 FPS trên CPU, YOLOv8 phù hợp nhất cho các ứng dụng realtime hoặc gần realtime.

Mô hình có kích thước rất nhỏ, khoảng 5.94 MB, giúp việc triển khai nhanh, nhẹ và ít tiêu tốn tài nguyên. Đây là lợi thế lớn khi xây dựng web application hoặc triển khai trên thiết bị có cấu hình hạn chế.

YOLOv8 cũng có quy trình huấn luyện tương đối đơn giản. Thư viện Ultralytics cung cấp API rõ ràng, hỗ trợ train, validation, inference và export model thuận tiện.

6.2.2 Nhược điểm

Mặc dù nhanh, YOLOv8 có độ chính xác thấp hơn Faster R-CNN trên tập test. mAP@0.5 của YOLOv8 là 0.3764, thấp hơn đáng kể so với Faster R-CNN.

YOLOv8 có thể gặp khó khăn hơn trong các trường hợp đối tượng nhỏ, bị che khuất hoặc nằm trong khung cảnh phức tạp. Do là one-stage detector, mô hình ưu tiên tốc độ nên trong một số trường hợp độ chính xác định vị bounding box có thể thấp hơn two-stage detector.

6.2.3 Khả năng áp dụng thực tế

YOLOv8 là lựa chọn tốt nếu mục tiêu chính là tốc độ xử lý và khả năng triển khai thực tế. Nếu ứng dụng yêu cầu phản hồi nhanh, ví dụ như webcam detection hoặc xử lý video, YOLOv8 phù hợp hơn Faster R-CNN.

Tuy nhiên, trong project này, do tiêu chí lựa chọn chính là độ chính xác trên tập test, YOLOv8 không được chọn làm mô hình cuối cùng.

6.3 Deformable DETR

6.3.1 Ưu điểm

Deformable DETR là kiến trúc hiện đại dựa trên Transformer. Mô hình có khả năng học quan hệ toàn cục giữa các vùng trong ảnh, đây là lợi thế so với các mô hình CNN truyền thống.

So với DETR gốc, Deformable DETR cải thiện hiệu quả attention bằng cách chỉ tập trung vào một số điểm quan trọng, giúp giảm chi phí tính toán và cải thiện khả năng hội tụ.

Kiến trúc này có tiềm năng tốt nếu được huấn luyện với lượng dữ liệu lớn, số epoch đủ dài và phần cứng phù hợp.

6.3.2 Nhược điểm

Trong thực nghiệm hiện tại, Deformable DETR có kết quả thấp nhất. Mô hình chỉ đạt mAP@0.5 là 0.1604 và mAP@0.5:0.95 là 0.0744. Precision, recall và F1-score đều thấp hơn đáng kể so với hai mô hình còn lại.

Một nguyên nhân có thể là mô hình Transformer-based thường cần nhiều dữ liệu và thời gian huấn luyện hơn để đạt hiệu quả tốt. Với dữ liệu traffic subset và số epoch giới hạn, mô hình chưa khai thác được hết tiềm năng.

Ngoài ra, Deformable DETR có độ phức tạp huấn luyện cao hơn. Việc cấu hình processor, định dạng annotation, mapping class và tối ưu hyperparameter phức tạp hơn YOLOv8.

6.3.3 Khả năng áp dụng thực tế

Trong điều kiện thực nghiệm của project, Deformable DETR chưa phù hợp để triển khai thực tế. Mô hình có độ chính xác thấp và tốc độ xử lý trên CPU cũng chưa đủ tốt.

Tuy nhiên, kiến trúc này vẫn có giá trị nghiên cứu vì đại diện cho hướng tiếp cận Transformer-based trong object detection. Nếu có thêm dữ liệu, GPU mạnh và thời gian huấn luyện dài hơn, Deformable DETR có thể được cải thiện.

7 Lựa chọn mô hình cuối cùng

Dựa trên kết quả thực nghiệm, Faster R-CNN là mô hình đạt độ chính xác cao nhất trong ba mô hình được thử nghiệm. Mô hình đạt mAP@0.5 là 0.6062 và mAP@0.5:0.95 là 0.3727, vượt trội so với YOLOv8 và Deformable DETR.

Mặc dù YOLOv8 có tốc độ xử lý nhanh hơn và phù hợp hơn với các ứng dụng realtime, project này ưu tiên lựa chọn mô hình có độ chính xác cao nhất trên tập test. Vì vậy, nhóm chọn Faster R-CNN làm mô hình chính để tích hợp vào web application.

Tuy nhiên, khi triển khai, cần lưu ý nhược điểm về tốc độ xử lý của Faster R-CNN. Trên CPU, mô hình chỉ đạt khoảng 0.37 FPS. Do đó, để sử dụng hiệu quả trong thực tế, hệ thống nên được chạy trên GPU hoặc áp dụng thêm các kỹ thuật tối ưu như giảm

kích thước ảnh đầu vào, tối ưu model hoặc chuyển sang mô hình nhẹ hơn nếu yêu cầu realtime.

8 Xây dựng ứng dụng web

Sau khi lựa chọn Faster R-CNN, nhóm xây dựng một web application cho phép người dùng tương tác với mô hình một cách trực quan. Ứng dụng hỗ trợ hai chế độ đầu vào:

- Upload ảnh từ máy tính.
- Sử dụng webcam để phát hiện đối tượng.

Luồng xử lý của ứng dụng như sau:

1. Người dùng chọn ảnh hoặc bật webcam.
2. Ảnh đầu vào được gửi đến backend.
3. Backend load mô hình Faster R-CNN và thực hiện inference.
4. Kết quả dự đoán gồm class name, confidence score và bounding box.
5. Ứng dụng vẽ bounding box lên ảnh hoặc trực tiếp lên khung webcam.
6. Người dùng xem kết quả trên giao diện web.

Ứng dụng web giúp minh họa khả năng áp dụng thực tế của mô hình đã huấn luyện. Tuy nhiên, do Faster R-CNN có tốc độ xử lý chậm trên CPU, chế độ webcam không đạt realtime hoàn toàn mà thực hiện phát hiện theo từng khoảng thời gian nhất định.

9 Kết luận

Trong đồ án này, nhóm đã xây dựng pipeline object detection hoàn chỉnh, bao gồm chuẩn bị dữ liệu, chia tập train/validation/test, huấn luyện nhiều mô hình, đánh giá kết quả và triển khai mô hình vào web application.

Dữ liệu được chia theo tỉ lệ 70/15/15, tương ứng với 3500 ảnh train, 749 ảnh validation và 750 ảnh test. Ba mô hình được thử nghiệm gồm Faster R-CNN, YOLOv8 và Deformable DETR.

Kết quả thực nghiệm cho thấy Faster R-CNN đạt độ chính xác cao nhất, YOLOv8 đạt tốc độ xử lý nhanh nhất, còn Deformable DETR chưa đạt hiệu quả tốt trong điều kiện huấn luyện hiện tại.

Dựa trên tiêu chí độ chính xác, nhóm lựa chọn Faster R-CNN làm mô hình cuối cùng. Mô hình này phù hợp với các bài toán yêu cầu độ chính xác cao, nhưng cần được tối ưu thêm nếu muốn triển khai trong các hệ thống realtime.

Trong tương lai, project có thể được cải thiện bằng cách mở rộng dữ liệu huấn luyện, tăng số epoch, thử nghiệm thêm các phiên bản YOLO lớn hơn, tối ưu Faster R-CNN để tăng tốc độ inference hoặc triển khai mô hình trên GPU để cải thiện hiệu năng của ứng dụng web.